DNA Sequence Reconstruction Lab

Oct 21, 2026 · @Alexandra

Exercise request

1. Take a DNA sequence of an entire genome from NCBI.
2. Take 2000 random samples (reads) from the whole sequence, each about 100–120 bases long.
3. Store the samples in an array or list.
4. Rebuild the DNA sequence using only these random samples.
5. Find out what the main problem is when rebuilding without an algorithmic approach.

Deliverables: a Python script (`dna_reconstruction.py`) and this document, which we update as the lab progresses.

Data used

Genome: SARS-CoV-2 isolate USA/PC00101P/2020, NCBI GenBank MT192765.1, complete genome, 29,829 bases. A small viral genome keeps the run fast and makes 2000 reads meaningful: 2000 × ~110 bases ≈ 220,000 bases, about 7.4× coverage.

To get it: open the NCBI link → Send to → File → FASTA, save as `genome.fasta` next to the script.

Python solution

The script `dna_reconstruction.py` runs in under a second and uses only the standard library.

1. `read_fasta` loads the genome as one string of A, C, G, T.
2. `sample_reads` picks 2000 random start positions and lengths (100–120) and stores the substrings in a list. A fixed seed (42) makes runs reproducible.
3. `naive_rebuild` joins the reads in the order we have them, with no algorithm.
4. `coverage_gaps` checks which genome bases no read covered (uses the true positions only for evaluation).
5. `greedy_assemble` is a comparison baseline: it joins reads by their longest suffix–prefix overlaps (minimum 20 bases) into contigs.

Results

The naive rebuild is 7.4× too long and matches the genome no better than chance; a simple overlap algorithm recovers it as 5 correct pieces.

Measure	Naive concatenation	Greedy overlap assembly
Rebuilt length (bases)	220,642	29,855 in 5 contigs
Real genome length (bases)	29,829	29,829
Matches the real genome	26.6% of positions (chance ≈ 25%)	all 5 contigs are exact pieces
Largest correct piece (bases)	~110 (one read)	8,524 (28.6% of genome)

Other figures: bases never sampled = 29 (0.10%), in 2 gaps at the genome's two ends; possible read orders = 2000! ≈ 10^5736; pairwise comparisons for overlaps = 3,998,000.

Main problem without an algorithm

The main problem is that the reads carry no position: we have 2000 overlapping pieces in random order and no way to know where each belongs or how much it overlaps its neighbours.

- **Lost order.** Joining reads as they come gives a 220,642-base string that matches the genome only at chance level.
- **Overlaps duplicate bases.** Even in the correct order, plain concatenation repeats every shared stretch, so the result stays 7.4× too long. The overlaps must be detected and merged.
- **Brute force is impossible.** Trying every order means 2000! ≈ 10^5736 permutations.
- **Gaps can't be filled.** Random sampling leaves some bases uncovered (29 here), so the genome usually comes out as several contigs, not one piece.
- **Repeats confuse joining.** Identical stretches in the genome make a read fit in more than one place; greedy joining can then pick the wrong neighbour.

The fix is an algorithmic approach that uses overlaps: greedy overlap merging, overlap–layout–consensus, or de Bruijn graphs.

Lab log

Date	Change	Result
2026-10-21	First version: sampling, naive rebuild, problem analysis, greedy baseline	5 exact contigs, largest 8,524 bases

- [ ] Next step: to be added as the lab continues